Étude de marché et choix technologiques – App d'épargne par arrondis

Nov 7, 2026 · @Timothée Benchergui

Résumé et décisions

Le projet est réalisable avec une stack Go, PostgreSQL et Vue 3 en application web progressive hébergée sur le Mac mini, à une condition : chaque utilisateur doit disposer de sa propre application Enable Banking, car les conditions de ce service interdisent de lire avec une même application des comptes qui ne sont pas ceux de son créateur.

Domaine	Décision	Plan B
Accès bancaire	Enable Banking, production restreinte, une application par utilisateur	Import manuel d'un relevé exporté de sa banque
Backend	Go	NestJS
Base de données	PostgreSQL 18	SQLite
Front	Vue 3 + Vite + TypeScript, PWA	React
Hébergement	Mac mini, Docker Compose, Portainer	Aucun
Exposition	Cloudflare Tunnel + Cloudflare Access	Aucun
Distribution	PWA installée sur l'écran d'accueil, aucune publication sur les magasins d'applications	Application native plus tard, sans toucher à l'API

Prochaine étape. La preuve de concept décrite dans le cahier des charges tranche les deux risques majeurs (R1 et R2) avant tout autre développement. Si BoursoBank, la banque de l'auteur, n'est pas disponible dans le panneau de contrôle d'Enable Banking, le projet s'arrête ou passe à l'import manuel. Si seules les trois autres banques manquent, elles restent désactivées et le projet continue (R2).

Contexte et objectifs

L'application calcule, pour chaque achat par carte du compte bancaire de l'utilisateur, la somme à mettre de côté en arrondissant le montant, puis cumule ces sommes dans un tableau de bord. Elle est strictement privée : son auteur et les quelques personnes qu'il invite. Elle est utilisable avec plusieurs banques : BoursoBank, Crédit Agricole, Crédit Mutuel et La Banque Postale.

- **Utilisateurs** : l'auteur et quelques proches invités, sur iPhone ou Android.
- **Fonction** : afficher le total épargné, l'épargne du mois, l'historique des arrondis et la progression vers un objectif d'épargne.
- **Aucun paiement** : l'application calcule le montant ; le virement vers l'épargne reste manuel, une fois par mois.
- **Budget** : 0 € de licence, hébergement sur un Mac mini déjà possédé (Docker et Portainer).
- **Distribution** : aucune publication sur l'App Store ni sur le Play Store.

Trois contraintes structurent tous les choix qui suivent :

1. **DSP2** : lire des comptes bancaires par API exige d'être agréé ou de passer par un agrégateur ; la section « Faisabilité de l'accès bancaire » traite ce point.
2. **Règle Apple 5.1.1(ix)** : les apps de services très réglementés, dont la banque et les services financiers, doivent être soumises par une entité juridique et non par un particulier (MacRumors, Apple). Une publication publique est donc exclue, d'où le choix d'une PWA.
3. **Données sensibles** : l'historique des achats bancaires de plusieurs personnes. L'authentification est déléguée à Cloudflare Access et les sauvegardes sont chiffrées.

Marché existant

Les services d'arrondis existants sont soit intégrés à une néobanque dont il faut devenir client, soit proposés par une application tierce sous contrat avec un agrégateur bancaire. Je n'ai trouvé aucune offre d'arrondis native chez BoursoBank, Crédit Agricole, Crédit Mutuel ou La Banque Postale dans les sources consultées (à revérifier avant de figer le périmètre).

Offre	Type	Mécanisme d'arrondi	Destination de l'épargne	Limite pour ce projet
Revolut	Néobanque	Arrondi à l'unité supérieure, multiplicateurs ×1 à ×10, règle propre à chaque devise	Pocket, cryptomonnaies ou RevPoints	Exige un compte Revolut
Trade Republic	Néobanque et courtier	Arrondi à l'euro supérieur, multiplicateurs ×2, ×3, ×5, ×10 ; cumul hebdomadaire, investi les 2, 9, 16 et 23 du mois	Action ou ETF choisi	Exige une carte Trade Republic
Moka (ex-Mylo)	Application d'épargne	Transactions récupérées via Budget Insight (groupe Arkéa) ; dépôt hebdomadaire	Épargne Moka	Service tiers sous contrat avec un agrégateur
Bitstack	Application bitcoin	Arrondis cumulés sur la semaine, prélevés en fin de semaine, convertis en bitcoin le dimanche	Bitcoin	Épargne uniquement en cryptomonnaie
helios	Banque en ligne	Arrondi à l'euro supérieur, disponible depuis mai 2025	Compte helios	Exige un compte helios

Ce que le projet reprend de ces offres : l'arrondi à l'unité supérieure, le multiplicateur et le cumul sur une période. Ce qu'il ajoute : des paliers au choix (1, 2, 5 ou 10 €, ou une valeur personnalisée) appliqués aux achats par carte d'un compte qui reste celui de l'utilisateur, chez BoursoBank, Crédit Agricole, Crédit Mutuel ou La Banque Postale, sans changer de banque ni payer d'abonnement.

Le projet ne cherche pas à concurrencer ces offres : c'est un outil privé pour quelques personnes. Il n'y a donc ni étude de taille de marché ni modèle économique à établir.

Faisabilité de l'accès bancaire

Enable Banking en mode production restreint est la seule piste gratuite et sans contrat retenue ; elle couvre vraisemblablement les quatre banques visées (BoursoBank, Crédit Agricole, Crédit Mutuel, La Banque Postale), à vérifier une par une (R2), mais ses conditions d'utilisation interdisent qu'une seule application lise les comptes de plusieurs personnes. Ce dernier point impose une architecture par utilisateur (décision D-01 du cahier des charges).

Point	Constat	Statut
Mode restreint	Une application de production s'active en liant ses propres comptes dans le panneau de contrôle, sans contrat ; seuls les comptes liés renvoient des données (Open Banking Tracker, DEV)	Confirmé
Usage autorisé	La production est limitée aux comptes liés et réservée à l'évaluation ou à l'usage personnel de particuliers ; le nombre de comptes liés par application peut être limité (conditions d'utilisation)	Confirmé
Comptes de tiers	Les conditions excluent l'accès aux informations de comptes qui n'appartiennent pas à l'utilisateur du panneau de contrôle ayant lié ces comptes (conditions d'utilisation)	Confirmé, bloquant pour une application unique

Point	Constat	Statut
Prise en charge des banques	Boursorama (FR) figure dans les intégrations corrigées et améliorées du journal des changements de février 2024 et dans la liste des connecteurs de la documentation ; aucune source récente (2026) n'a été trouvée pour BoursoBank. La page de couverture de la France d'Enable Banking liste aussi Crédit Agricole (avec le choix de la caisse régionale), Crédit Mutuel et La Banque Postale. Chaque banque reste à valider sur un compte réel	À vérifier dans le panneau de contrôle (étape 1 du plan)
Fréquence de lecture	Les banques plafonnent à 4 lectures par jour sans présence de l'utilisateur, selon la FAQ d'Enable Banking rapportée par Anchor Terminal	Confirmé par une source secondaire
Durée du consentement	Une autorisation DSP2 dure en général 180 jours avant ré-autorisation (TrackMyWealth) ; la durée exacte n'est pas documentée pour chacune des quatre banques	À vérifier pendant la preuve de concept

Conséquence de la clause sur les comptes de tiers. Chaque utilisateur doit créer son propre compte Enable Banking, enregistrer sa propre application et lier son propre compte bancaire. L'API stocke alors, par utilisateur, l'identifiant d'application et la clé privée (chiffrée). Une application commune lisant les comptes de plusieurs personnes serait contraire aux conditions et pourrait être désactivée par Enable Banking.

Solution de repli si cette contrainte est jugée trop lourde pour certains proches : import manuel d'un relevé exporté depuis l'espace client de la banque. La disponibilité et le format de cet export doivent être vérifiés avant de le promettre.

Pistes écartées.

- Tink : BoursoBank y est listée avec une connexion désactivée (Fintable).

- GoCardless Bank Account Data : inscriptions fermées (recherche antérieure du projet).
- Powens et Bridge : offres réservées aux entreprises sous contrat (recherche antérieure du projet).

Choix du backend : Go

Go obtient 63 points sur 70, contre 46 pour NestJS, son suivant. Les notes ci-dessous sont une évaluation subjective de l'auteur de 0 à 5, pondérée par l'importance du critère pour ce projet ; elles servent à rendre la décision traçable, pas à la démontrer.

Critère (poids)	Go	NestJS	FastAPI	Spring Boot	Laravel
Empreinte sur le Mac mini (3)	5	3	3	1	3
Typage et fiabilité des montants (3)	5	3	3	5	2
Simplicité de déploiement (3)	5	3	3	2	3
Maintenance sur plusieurs années (2)	5	3	3	4	3
Vitesse de développement (2)	3	4	5	3	5
Types partagés avec le front (1)	2	5	2	2	2
Total sur 70	**63**	**46**	**45**	**40**	**42**

Les ordres de grandeur utilisés pour l'empreinte sont des estimations de l'auteur, non mesurées : environ 15 à 20 Mo d'image Docker et quelques dizaines de Mo de mémoire pour Go ; environ 150 Mo de mémoire pour NestJS ; 300 à 500 Mo pour Spring Boot (JVM). Elles doivent être mesurées pendant la preuve de concept.

Techno	Points forts	Points faibles	À choisir plutôt que les autres si…	À éviter si…
Go	Binaire unique, image minimale, typage strict, `int64` natif pour les centimes, tâches planifiées dans le processus	Plus verbeux, gestion d'erreurs répétitive, pas de types partagés (OpenAPI à la place)	le déploiement le plus léger et un service qui tourne des années sans entretien comptent le plus	on veut écrire le moins de code possible ou garder un seul langage front et back

Techno	Points forts	Points faibles	À choisir plutôt que les autres si…	À éviter si…
NestJS	Même langage que le front, types partagés, écosystème très large	`node_modules` lourd, dépendances à surveiller, montants en `number` flottant	un seul langage partout prime sur la légèreté	on veut un serveur minimaliste : sa chaîne de dépendances est la plus lourde pour ce volume
FastAPI	Très rapide à écrire, OpenAPI natif	Typage dynamique, déploiement Python plus fragile	une preuve de concept doit sortir très vite	le service doit tourner sans surveillance
Spring Boot	Très mature, typage fort	JVM gourmande, démarrage lent, beaucoup de configuration	le projet doit grossir (équipe, modules) ou sert d'entraînement au cours JEE	on héberge sur une petite machine personnelle
Laravel	Migrations, planificateur et ORM intégrés	Image plus lourde (PHP-FPM et nginx), typage plus souple	la rapidité de mise en place prime	l'authentification est déjà déléguée, ce qui supprime son avantage principal

Décision. Go, avec NestJS comme plan B si un seul langage pour tout le projet devient prioritaire. Le langage n'est pas un critère : l'auteur maîtrise Spring, FastAPI, Go, PHP et NestJS.

Choix de la base de données : PostgreSQL

PostgreSQL obtient 49 points sur 55, contre 41 pour SQLite et MariaDB. Le critère décisif est la marge d'évolution, que l'auteur a explicitement choisi de privilégier : il ne veut pas se priver de pouvoir faire grandir le projet.

Critère (poids)	PostgreSQL	SQLite	MariaDB / MySQL	MongoDB
Intégrité des données : types, contraintes, transactions (3)	5	4	4	2

Critère (poids)	PostgreSQL	SQLite	MariaDB / MySQL	MongoDB
Marge d'évolution (3)	5	2	4	3
Simplicité d'exploitation (2)	3	5	3	3
Support Go avec `sqlc` (2)	5	4	4	1
Empreinte sur le Mac mini (1)	3	5	3	2
Total sur 55	**49**	**41**	**41**	**25**

Sensibilité : si le poids de la marge d'évolution passe de 3 à 1, PostgreSQL reste devant (39 sur 45) mais SQLite le talonne (37 sur 45). Le choix de PostgreSQL repose donc sur la volonté de pouvoir évoluer, pas sur un avantage écrasant aujourd'hui.

Base	Points forts	Points faibles	À choisir plutôt que les autres si…	À éviter si…
PostgreSQL	Types riches (`timestamptz`, `JSONB`), contraintes strictes, transactions solides, évolutif	Un conteneur de plus, mots de passe et sauvegardes à gérer	on veut la base la plus solide avec de la marge pour grossir	on cherche la simplicité maximale pour 5 utilisateurs
SQLite	Un fichier, aucun conteneur, sauvegarde par copie, démarrage instantané	Un seul écrivain à la fois, pas d'accès réseau natif, peu de types	on veut le déploiement le plus simple possible	plusieurs instances de l'API doivent écrire en même temps
MariaDB / MySQL	Très répandue, mature	Moins strict par défaut, dates plus fragiles	on s'intègre à un existant qui l'utilise	on part de zéro : PostgreSQL fait la même chose en mieux
MongoDB	Schéma souple	Pas de jointures simples, pas de `sqlc`	les données sont des documents sans structure fixe	les données sont des tables liées (utilisateurs, comptes, transactions, arrondis)

Décision. PostgreSQL 18, version majeure stable en juin 2026 avec la mineure 18.4 (Instaclustr), prise en charge jusqu'en novembre 2030. Accès avec `pgx` et `sqlc`, migrations avec `goose`, sans ORM. SQLite reste le plan B si l'on préfère retirer le conteneur de base.

Choix du front : Vue 3

Vue obtient 49 points sur 55, contre 47 pour React et 44 pour Svelte. L'écart avec React est faible : le choix de Vue repose sur la simplicité (routeur et gestion d'état officiels) et non sur un avantage écrasant.

Critère (poids)	Vue	React	Svelte	Angular	Go + HTMX
Maturité PWA : installation, cache, mises à jour (3)	5	5	4	4	2
Pérennité et écosystème (3)	4	5	3	4	3
Simplicité pour une petite application (2)	5	3	5	2	4
Poids du bundle (1)	4	3	5	2	5
Client d'API typé généré depuis l'OpenAPI (2)	4	4	4	5	3
Total sur 55	**49**	**47**	**44**	**40**	**34**

Techno	Points forts	Points faibles	À choisir plutôt que les autres si…	À éviter si…
Vue 3 + Vite	Syntaxe claire, Pinia et Vue Router officiels, support PWA mature	Écosystème un peu plus petit que React	on veut le meilleur équilibre simplicité et écosystème pour une PWA	on veut le plus petit bundle possible ou l'écosystème le plus vaste
React + Vite	Écosystème gigantesque, nombreuses ressources	Plus verbeux, il faut choisir ses briques (état, routeur, formulaires)	on envisage plus tard une application React Native	on veut un framework qui impose moins de choix
SvelteKit	Bundle minuscule, très rapide, peu de code	Écosystème plus petit, API ayant changé avec Svelte 5	on veut le moins de code possible	on compte sur beaucoup de composants tiers
Angular	Framework complet et structuré	Lourd, beaucoup de cérémonie	le projet devient gros et collaboratif	c'est une petite application à un seul développeur
Go + HTMX	Aucun build front, un seul langage	Pas de vrai mode hors ligne, effet « application » limité sur iPhone	on veut zéro JavaScript côté projet	on veut transitions, état local et hors ligne, ce qui est l'enjeu d'une PWA

Le plugin PWA retenu est `vite-plugin-pwa`, compatible Vue 3, React et Svelte et fondé sur Workbox (dépôt officiel).

Contrainte propre aux PWA sur iPhone. Safari efface les données d'un site (IndexedDB, cache, service worker) après 7 jours sans interaction, mais pas celles d'une application ajoutée à l'écran d'accueil en affichage plein écran (WebKit, web.dev). Le manifeste doit donc déclarer `display: standalone` et chaque utilisateur doit installer l'application sur son écran d'accueil. Par prudence, la source de vérité reste la base côté serveur ; le mode hors ligne se limite à un cache de lecture.

Décision. Vue 3 + Vite + TypeScript, avec Pinia, Vue Router, Tailwind CSS et un client généré par `openapi-typescript`. React est le plan B si une application React Native est envisagée.

Stack définitive

La stack tient en trois blocs (API Go, base PostgreSQL, PWA Vue) déployés par Docker Compose sur le Mac mini. Les versions de bibliothèques sont épinglées au démarrage dans `go.mod` et dans le fichier de verrouillage du front ; seules les versions vérifiées sont données ici.

Couche	Technologie	Rôle	Version
Langage backend	Go	API, synchro bancaire, moteur d'arrondis	Branche 1.26 (stable ; 1.27 en préversion en juin 2026, versionlog)
Serveur HTTP	`net/http` standard	Routage et service de l'API	Fourni avec Go
Description d'API	`huma`	Génération du contrat OpenAPI	Dernière stable au démarrage
Accès base	`pgx` + `sqlc`	Requêtes SQL écrites à la main, code Go généré	Dernières stables au démarrage
Migrations	`goose`	Versionnement du schéma	Dernière stable au démarrage
Tâches planifiées	`robfig/cron`	Synchro bancaire quotidienne dans le processus	Dernière stable au démarrage
JWT	`go-jose`	Vérification du JWT Cloudflare Access ; signature RS256 pour Enable Banking	Dernière stable au démarrage

Couche	Technologie	Rôle	Version
Base de données	PostgreSQL	Stockage	18 (mineure 18.4 en juin 2026, Instaclustr)
Front	Vue 3 + Vite + TypeScript	Application web progressive	Dernières stables au démarrage
PWA	`vite-plugin-pwa`	Manifeste, service worker, mises à jour	Série 1.x (dépôt)
État et navigation	Pinia, Vue Router	État de l'application, écrans	Dernières stables au démarrage
Style	Tailwind CSS	Mise en forme	Dernière stable au démarrage
Client d'API	`openapi-typescript`	Types TypeScript générés depuis l'OpenAPI	Dernière stable au démarrage
Conteneurs	Docker Compose, Portainer	Déploiement de la stack	Déjà installés sur le Mac mini
Exposition	Cloudflare Tunnel (`cloudflared`) + Cloudflare Access	Accès Internet sans ouvrir de port, authentification par email	Service Cloudflare
Sauvegardes	`pg_dump` + `age`	Dump quotidien chiffré, copié hors du Mac mini	Dernières stables au démarrage
Accès bancaire	Enable Banking, mode production restreint	Lecture des transactions des banques visées	API service

Services Docker Compose. `db` (PostgreSQL 18, volume persistant), `api` (binaire Go dans une image minimale), `web` (nginx servant les fichiers statiques de la PWA), `cloudflared` (tunnel). Aucun port n'est publié sur la box : le trafic entre uniquement par le tunnel Cloudflare. Les ports 80 et 443 de la box restent réservés aux portfolios publics.

Organisation du code. Un seul dépôt Git avec deux dossiers : `api/` (Go) et `web/` (Vue). Le client TypeScript est régénéré à chaque changement du contrat OpenAPI.

Risques et points ouverts

Deux risques peuvent arrêter ou remodeler le projet : la clause d'Enable Banking sur les comptes de tiers (R1) et la prise en charge réelle des quatre banques en 2026 (R2). Les deux se tranchent avant tout développement, par la preuve de concept.

Id	Risque	Probabilité	Impact	Mesure
R1	Une application Enable Banking ne peut lire que les comptes de la personne qui l'a créée (conditions d'utilisation)	Certaine	Élevé	Architecture par utilisateur : chaque utilisateur a son compte et son application Enable Banking ; repli par import manuel d'un relevé
R2	Une banque n'est plus prise en charge, ou l'est mal	Faible à moyenne	Bloquant	Vérifier dans le panneau de contrôle d'Enable Banking avant d'écrire du code métier ; sinon arrêt ou import manuel
R3	Limites du mode restreint non publiées (nombre de comptes liés, fréquence de lecture)	Moyenne	Moyen	Mesurer pendant la preuve de concept ; une seule synchro par jour, bien en deçà du plafond de 4 lectures
R4	Expiration du consentement DSP2 (environ 180 jours)	Certaine	Faible	Alerte 14 jours avant l'expiration et écran de ré-autorisation
R5	Perte des données sur le Mac mini	Faible	Élevé	Dump chiffré quotidien copié hors site ; test de restauration chaque trimestre
R6	Compromission de l'accès (tunnel, Cloudflare Access)	Faible	Élevé	Liste d'emails autorisés, JWT vérifié côté API, secrets chiffrés, journalisation des accès
R7	Enable Banking supprime le mode restreint ou le rend payant	Faible	Élevé	Isoler l'accès bancaire derrière une interface interne pour pouvoir changer de fournisseur

Id	Risque	Probabilité	Impact	Mesure
R8	L'empreinte mémoire réelle dépasse les estimations	Faible	Faible	Mesurer pendant la preuve de concept

Points ouverts à trancher par l'auteur. Ils sont numérotés D-xx et proposés avec une valeur par défaut dans la section « Décisions à valider » du cahier des charges.

Sources

Recherche effectuée le 7 novembre 2026. Les extraits viennent des résultats de recherche ; la page des conditions d'utilisation d'Enable Banking n'a pas pu être ouverte en entier (elle exige JavaScript), le passage cité est celui du résultat de recherche. Les notes de comparaison (backend, base, front) sont des évaluations de l'auteur et ne viennent d'aucune source.

Accès bancaire

- Enable Banking, conditions d'utilisation
- Enable Banking, journal des changements de février 2024 (Boursorama)
- Enable Banking, documentation du cœur d'agrégation (connecteur Boursorama)
- Open Banking Tracker, API ouvertes gratuites
- DEV, accès de production des fournisseurs DSP2
- Anchor Terminal, fiche Enable Banking
- TrackMyWealth, étude Enable Banking
- Fintable, BoursoBank via Tink

Marché

- Revolut, arrondis
- Trade Republic, Round up
- ConnectBanque, avis sur Moka
- S'investir, avis sur Bitstack
- Comparateur Banque, arrondi chez helios

Règles de plateformes et versions

- MacRumors, règle Apple 5.1.1(ix)
- Apple, App Review
- WebKit, stockage des applications à l'écran d'accueil
- web.dev, stockage pour le web
- Versionlog, Go

- Instaclustr, versions de PostgreSQL
- vite-plugin-pwa, dépôt officiel